

[INTERNAL] How to try out Lakeflow Connect Zerobus

[Step 1] Create a table

1. Log into [Dogfood workspace](#) and locate or create a table for ingesting. You can use notebook for creating a table

If you don't have a catalog, you can use *shinkansen*.

Note: We support only managed tables.

Example:

Sample table created with Notebook.

```
None
%sql
CREATE TABLE shinkansen.default.test_table(
  id INT, name String, submit_counter LONG
);
```

2. [[Dogfood](#)] Navigate on "Catalog" using left-hand side panel -> navigate to your table -> permissions -> grant -> MODIFY, USE SCHEMA, USE CATALOG

[Step 2] Create a PAT in PrPr Workspace

3. Log into [PrPr-enabled Workspace](#).

You should get permissions for this workspace - ask somebody on *#lakeflow-connect-row-ingest*

IMPORTANT:

PrPr-enabled workspace is being used only for generating PAT token - don't create tables on it.

4. Issue a [PAT token](#) on PrPr-enabled workspace

Go to: Workspace -> Click user located on top right -> Settings -> Developer -> Access Token -> Manage -> Generate New Token

[Step 3] Create a Python Application on your laptop

5. Install Python SDK

Run the following:

```
# Download .whl from the link
> mkdir test_dir
# Move the .whl to the test_dir
> cd test_dir
> python3.9 -m venv venv39
> source venv39/bin/activate
> pip install databricks_ingest-0.0.10-py3-none-any.whl # align to the actual name if needed
```

Organizers have built the Python wheel (.whl extension) for you. You just need to install it with python*.

**Requirement: Python 3.9+ (should be preinstalled)*

6. Generate proto with Python wheel

```
> cd test_dir # If not already in the test_dir
> generate_proto --uc-endpoint
https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com --uc-token <PAT_TOKEN>
--table <catalog.schema.table> --output <proto_name>.proto
> generate_proto -h # To see all the arguments for the generate_proto command
```

<proto_name>.proto that was created should look something like this:

```
Protobuf
syntax = "proto2";
message <proto_name> {
    optional int32 id = 1;
    optional string name = 2;
    optional int64 submit_counter = 3;
}
```

7. Compile proto-message definition

```
> cd test_dir # Move to test_dir if not already there
> pip install grpcio-tools
> python3 -m grpc_tools.protoc -I. --python_out=. <proto_name>.proto
```

Python will generate `<proto_name>_pb2.proto` file, which you need to import into your Python script for ingesting.

8. Create a Python client for ingesting rows

```
> touch client.py # Create client file
> cursor client.py # Open with your IDE
```

- + Set up the constant variables such as PAT token generated

Example:

Create `<client_filename>.py` file

```
Python
import asyncio
# CHANGE IMPORT TO MATCH YOUR PROTO
import proto_name_pb2
from ingest_api_sdk import IngestApiSdk, TableProperties

# CHANGE CONSTANTS TO MATCH YOUR USECASE
DATABRICKS_WORKSPACE_URL =
"https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com"
TABLE_NAME = "shinkansen.default.test_table"
PAT = "<YOUR_TOKEN>" # IMPORT TOKEN
SERVER_ENDPOINT = "ingest.staging.cloud.databricks.com"

sdk_handle = IngestApiSdk(SERVER_ENDPOINT, DATABRICKS_WORKSPACE_URL, PAT)

async def blocking_call():
    # Create stream to table.
    # CHANGE DESCRIPTOR
    table_properties = TableProperties(TABLE_NAME,
proto_name_pb2.TestTable.DESRIPTOR)
    stream = await sdk_handle.create_stream(table_properties)

    # blocking call which completes when the message is durable.
    # INGEST ROWS
    record_future = await stream.ingest_record(
        proto_name_pb2.TestTable(id=1, name="John Doe", submit_counter=1)
    )
```

```
ack1 = await record_future
print(f"Record with ID: {ack1} persisted.")

# Close the stream
await stream.close()
print("Stream closed.")

if __name__ == "__main__":
    asyncio.run(blocking_call())
```

Note:

To run this specific file, change import to match your proto, but also change constants, descriptors and `ingesting_record()` method to match your desired table.

9. Run!

```
> python3 <client_filename>.py
```